

UNIT 1

Activity 1: Inverted Index Generation using MapReduce

CO Mapped: CO1 –Use Big Data and its Business Implications.

Type: Practical + Analytical Activity

Problem Statement

In Information Retrieval systems, an Inverted Index is a core data structure that maps each unique word to the list of documents in which it appears. This indexing technique enables fast search, efficient querying, and scalable document retrieval across large text collections. Large-scale search platforms such as Google and Microsoft rely on inverted indexing mechanisms to process billions of documents efficiently. In distributed environments, generating such an index requires parallel processing frameworks like Apache Hadoop, which implements the MapReduce model for large-scale data computation.

Your task is to design and implement a MapReduce program that processes a collection of text documents stored in HDFS and generates an Inverted Index. Each document will be stored as a separate text file, and the file name should be treated as the Document ID. The program must extract all unique words from the dataset and output each word along with the list of Document IDs in which it appears.

Objective

This activity strengthens student capability in designing distributed text-processing systems using MapReduce. The activity supports development of skills in key-value pair generation, data normalization, aggregation logic, and structured output formatting in Hadoop.

Tasks to Perform

Task 1: Hadoop Setup and Dataset Preparation

This task introduces the preparation of a multi-document dataset inside Hadoop. Students configure the environment and upload text files to HDFS.

- Install and configure Hadoop (Standalone or Pseudo-distributed mode)
- Create an input directory in HDFS
- Upload multiple .txt documents
- Ensure each file has a unique Document ID (file name)

Sample Input Dataset

Document 1 (Doc1.txt)

Hadoop enables distributed processing of large data sets.

Hadoop uses MapReduce for scalable data analysis.

Big data processing requires distributed systems.

Document 2 (Doc2.txt)

MapReduce is a programming model for processing large data sets.

Distributed systems improve scalability and reliability.

Data analysis is important in big data applications.

Task 2: Mapper Implementation

- Read text line by line
- Convert words to lowercase
- Remove punctuation and special characters
- Tokenize text into individual words
- Emit intermediate output in the format:

Output of Mapper : (word, DocumentID)

Task 3: Reducer Implementation

- Group all Document IDs for each word
- Remove duplicate Document IDs
- Format final output as:

Output of Reducer : word → Doc1.txt, Doc2.txt, Doc3.txt

Task 4: Execution and Verification

- Compile the MapReduce program
- Create a JAR file
- Execute the job in Hadoop
- Verify output stored in HDFS
- Capture screenshots of successful execution

Task 5: Written Summary Document

- Prepare a 500-word report
- Explain Mapper and Reducer logic
- Describe preprocessing steps
- Provide sample intermediate and final output
- Discuss challenges faced (duplicates, case normalization, formatting)

What Students Must Submit

Students must submit:

- Java/Python source code (Mapper, Reducer, Driver)
- Input dataset used
- Output file generated (part-r-00000)
- Screenshots of Hadoop execution
- 500-word written summary document
- Single compressed folder or GitHub repository link

Learning Outcome

Upon completion of this activity, students will be able to:

- Understand the role of Inverted Index in information retrieval systems
- Implement distributed data processing using MapReduce
- Perform text normalization and tokenization at scale
- Manage Hadoop job execution and output validation
- Document distributed computing workflows effectively

Rubric – Inverted Index using MapReduce

Task	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Poor (1 Mark)	No Attempt (0)
Hadoop Setup & Dataset Preparation	Correctly configures Hadoop environment, creates HDFS directories, uploads multiple documents, and ensures proper Document ID structure. Demonstrates clear understanding of distributed storage.	Configures Hadoop and uploads dataset with minor issues or limited explanation.	Basic setup completed but lacks proper structure or clarity in dataset preparation.	Setup incomplete or incorrectly configured; dataset not properly organized.	No setup or dataset preparation submitted.
Mapper Implementation (Word Extraction)	Implements an efficient Mapper that accurately performs lowercase conversion, punctuation removal, tokenization, and emits correct (word, DocumentID) pairs for all inputs.	Mapper implemented correctly with minor preprocessing or formatting issues.	Basic Mapper logic present but limited preprocessing or inconsistent output.	Mapper contains major logical errors or incorrect key-value emission.	No Mapper implementation submitted.
Reducer Implementation (Inverted Index Generation)	Reducer correctly aggregates unique Document IDs, removes duplicates, formats output clearly, and generates accurate inverted index results.	Reducer aggregates Document IDs correctly with minor formatting or duplication issues.	Basic aggregation implemented but incomplete duplicate removal or inconsistent formatting.	Reducer logic incorrect or fails to generate proper inverted index.	No Reducer implementation submitted.
Execution & Output Verification	Successfully compiles, packages, and executes job in Apache Hadoop using the MapReduce. Provides clear screenshots/logs and validates output in HDFS.	Executes job successfully with minor configuration or validation issues.	Job execution shown with limited proof or incomplete output verification.	Execution unsuccessful or poorly documented.	No execution evidence submitted.
Task 5: Written Summary Document (500 Words)	Produces a well-structured report clearly explaining problem statement, MapReduce workflow, preprocessing steps, intermediate output, final results, and challenges with strong analytical insight.	Report explains major steps and results clearly with minor gaps in depth.	Basic report with simple description and limited technical explanation.	Report incomplete, poorly structured, or lacking clarity.	No report submitted.

